Original Article

Institution of MECHANICAL ENGINEERS

Journal of MECHANICAL ENGINEERING SCIENCE

Proc IMechE Part C:
J Mechanical Engineering Science
227(6) 1224–1241

DOI: 10.1177/0954406212458202
pic.sagepub.com

SAGE

Sensor-based navigation of a car-like robot based on Bug family algorithms

Dong-Hyung Kim, Kyoosik Shin, Chang-Soo Han and Ji Yeong Lee

Abstract

This article presents a sensor-based navigation algorithm for 3-degree-of-freedom car-like robots with a nonholonomic constraint. Similar to the classical Bug family algorithms, the proposed algorithm enables the car-like robot to navigate in a completely unknown environment by only using range sensor information. The car-like robot uses the local range sensor view to determine the local path so that it moves toward the goal. To guarantee that the car-like robot can approach the goal, the two modes of motion are repeated, termed as motion-to-goal and wall-following. The motion-to-goal behavior lets the robot directly move toward the goal and the wall-following behavior makes the car-like robot circumnavigate the obstacle boundary until it meets the leaving condition. For each behavior, the nonholonomic motion for the car-like robot is planned in terms of the turning radius at each step. The proposed algorithm is implemented with a real robot and the experimental results show the performance of the proposed algorithm.

Keywords

Autonomous navigation, car-like robot, Bug algorithm, nonholonomic constraint, pure pursuit

Date received: 22 April 2012; accepted: 25 July 2012

Introduction

Autonomous navigation for car-like robots has been a challenging field of research. For these researches in robot navigation, the car-like robot must depend on the local information from the sensor during the navigation. Thus, the sensor-based navigation algorithm needs to guarantee that the robot approaches the goal location only using the local information of the environment. Another difficulty to navigate the car-like robot occurs because of the motion constraints, like the nonholonomic constraint preventing the side-slip motion in the perpendicular direction, and the minimum turning radius. In order to design the motion planner for the car-like robot, the nonholonomic motion generated from the navigation algorithm should satisfy these motion constraints and the global convergence to the goal.

This article proposes a sensor-based algorithm for the car-like robot based on the Bug family algorithms. For the point robot with a contact sensor or a zero-range sensor, the simplest navigation algorithms called Bug1, Bug2 algorithms were introduced,[1] whereas the Tangent Bug algorithm is used when the robot has a finite-range sensor with a 360° infinite orientation resolution.[2] Basically, the two behaviors, the motion-to-goal and the wall-following, are repeated until the robot reaches the goal or a loop is detected, in which case the goal is not reachable. The motion-to-goal moves the robot in the direction of the shortest path to the goal. As the robot avoids the obstacle, the distance to the goal may reach a local minimum and begin to increase. If this happens, the motion-to-goal is terminated and the wall-following is executed so that the robot circumnavigates the obstacle boundary until the distance to the goal becomes smaller than the recorded minimum distance to the goal. These works have been applied to various navigation tasks.[3,4] Extensions to classical Bug-like algorithms are introduced in many works.[3,5,6] The problem of the planetary rover navigation with a limited field of view (FOV) is addressed by the WedgeBug algorithm. This navigation algorithm minimizes the requirement to sense and store data using autonomous gaze control.[3] As a variant of TangentBug, CautiousBug was introduced in Magid and Rivlin.[5] Although the structure of the algorithm is the same as Bug-like algorithm, CautiousBug algorithm uses a cautious search when it determines the

Department of Mechanical Engineering, Hanyang University, Gyeonggi-do, Korea

Corresponding author:
Ji Yeong Lee, Department of Mechanical Engineering, Hanyang University, Gyeonggi-do, Korea.
Email: jiyeongl@hanyang.ac.kr

direction to follow on obstacle boundaries. Also, for on-line robot navigation, CBug algorithm was designed for the navigation of disc robot and its competitiveness in terms of path stability has been established.[6] However, the above works consider the navigation of robot without accounting for motion constraints. The minimum turning radius of robotic vehicle limits the instantaneous reachable space; thus, the convergence to goal point for the motion planning is not guaranteed.

Compared to the two degree-of-freedom (DOF) point robot, the car-like robot is modeled with three DOFs, including not only position but also orientation, and it also has a nonholonomic constraint. A number of approaches have been proposed recently to cope with this motion constraint for the car-like robot navigation. The mapping from the configuration space to the specific configuration space was suggested so that the well-known reactive collision avoidance method for a holonomic robot like a potential field can be applied to the nonholonomic robot in that space.[7] Thus, the transformation between two spaces is necessary for executing the reactive navigation method. Nonholonomic path deformation methods were introduced in Lamiraux et al.[8] which simultaneously execute the real-time path planning and obstacle avoidance. Given an initial path, the methods in Lamiraux et al.[8] define the potential field over the space of a path. The obstacle potential is generated from the obstacle and determines the direction of path deformation. Since these methods compute the obstacle potential field along the path, it requires a lot of computation by the robot's real-time controller. These reactive approaches also suffer from local minima by which the robot can be stuck in the same place.

Another work of the autonomous navigation describes the sensor-based navigation using the generalized Voronoi graph (GVG).[9] In this work, the initial path is found using the GVG, and then this path is smoothed using a Beizer curve so that it becomes feasible to follow for the car-like robot. Even though the smooth local planning is possible during the robot navigation, the processes of construction of GVG and the path smoothing require higher computational cost than the conventional GVG approach. In Lanzoni and Sanchez,[10] a motion planning method based on Lazy DRM and Lazy LRM is proposed to compute the collision-free and feasible path for the car-like robot. By implementing the local planner which finds the optimal paths for a car-like robot, the nonholonomic path for a car-like robot is correctly generated in the known free region. When the algorithm fails to find the path to goal in the known free region, it tries to scan to extend its known space, and then these steps are repeated until the algorithm finds the path to goal or concludes that the goal is not reachable from the current configuration of the robot. Unfortunately, unnecessary backward motion of the robot may be generated during the robot navigation which increases the path length.

In this article, we focus on the navigation algorithm which provides the nonholonomic path to a goal for a car-like robot that insures the global convergence using only locally available sensor information. The proposed navigation algorithm generates the local nonholonomic motion using the instant turning radius which satisfies the motion constraints on the robot. Also, the lower limit on the turning radius exists due to the maximum value of the steering angle.

As a general limitation of sensor-based methods, the car-like robot may not able to avoid the obstacle depending on the value of limited sensing range and FOV. For example, let us assume that the maximum sensing range is equal or smaller than the minimum turning radius. Even if the FOV is 360°, the car-like robot collides with the blocking obstacle without the backward motion in this case.

We consider only the forward motion of car-like robot to minimize travel time. Thus, the narrow passage which requires the backward motion is not allowable. In addition, the orientation at goal point is not used for the navigation algorithm. The robot stops at the goal point while maintaining the previous orientation.

To satisfy the collision-free motion during the navigation, this article proposes the conditions for correctness so that the local collision-free motion of the robot is guaranteed. Also, even though the notions of the motion-to-goal and the wall-following from the Bug family algorithms are analogously used, we also include the local feasible motion of the car-like robot to guarantee completeness. Therefore, it is straightforward to implement for a real robot without heavy computation to obtain the global nonholonomic path to the goal point.

The following section describes the 'sensor-based nonholonomic navigation algorithm' in detail, including the motion-to-goal and wall-following behaviors. Then, the 'analysis of algorithm' for providing the completeness and the correctness for the navigation is discussed. The 'experimental results' to validate the proposed navigation algorithm is shown, and finally, the 'conclusions' are presented.

Sensor-based navigation algorithm

In this section, we describe the sensor-based navigation algorithm for the car-like robots. The proposed algorithm navigates the car-like robot in a planar unknown environment populated by stationary obstacles. The car-like robot is a point robot with the motion constraints, a nonholonomic constraint and a minimum turning radius. Only the forward motion is allowed to the robot during the navigation. We assume that the robot is equipped with a range sensor with a limited sensing range.

Let us briefly represent the sensor-based navigation algorithm. The proposed navigation algorithm uses the two basic behaviors, the motion-to-goal and wall-following. For each behavior, the instantaneous nonholonomic motion is generated in terms of the turning radius using the pure pursuit method[11,12] so that the motion constraints of the robot are satisfied. That is, the sequence of turning radius is used to find the nonholonomic path to the goal point.

During the motion-to-goal behavior, the robot moves toward the goal point by following the nonholonomic path. The robot first finds the endpoints of the obstacle boundaries within the sensing range. Then, it selects an endpoint which minimizes the distance to the goal, and the endpoint is regarded as an instantaneous goal. Therefore, the nonholonomic path to the instantaneous goal is constructed using the pure pursuit method. This enables the robot to move toward the instantaneous goal until the obstacle does not intersect the path to the final goal or the distance to the goal point begins to increase.

When the distance to the goal point begins to increase during the motion-to-goal behavior, the local minima is determined and the behavior switches to the wall-following behavior. The robot circumnavigates the obstacle boundary until the current distance to the goal becomes smaller than the recorded minimal distance to the goal point. Then, the robot leaves the obstacle boundary by switching the behavior to the motion-to-goal.

Since the car-like robot cannot move directly in the locally optimal direction that decreases the distance to the goal, the Euclidean distance to the goal can increase at the beginning of the motion-to-goal behavior. However, this should be distinguished from the local minima during the wall-following. Therefore, such a local path segment is defined as the motion-to-goal segment if the robot does not meet the obstacle. Otherwise, the robot should avoid the obstacle by following the obstacle boundary. Thus, the local path segment becomes the wall-following segment.

Basic notions

Kinematics model of the car-like robot. In Figure 1, the global coordinate system is a fixed coordinate system, and each axis is denoted by the superscript, G. The robot coordinate system, denoted by a superscript R, is a moving coordinate system attached to the robot, with the x-axis aligned with the heading of the robot. If the position and the orientation of the car-like robot are expressed by (x, y) and θ, respectively, the configuration of the car-like robot can be defined as $\mathbf{q} = [x\ y\ \theta]^{\mathrm{T}}$. Therefore, the kinematic model of the car-like robot is expressed by

$$\dot{\mathbf{q}} = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} \mathbf{u} \tag{1}$$

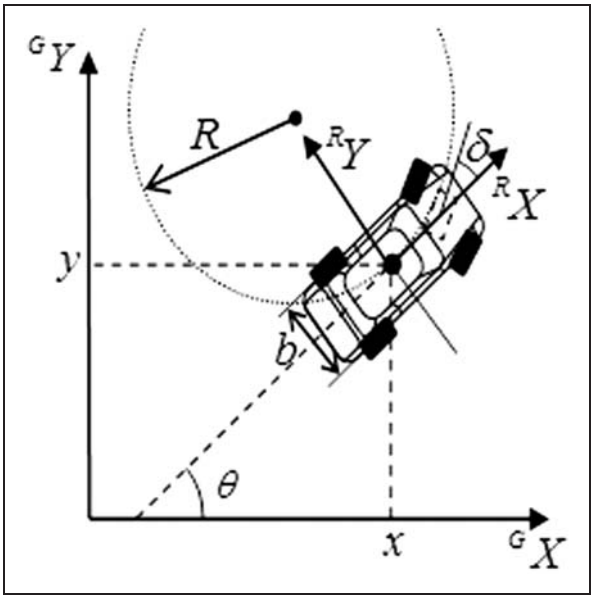

Figure 1. Kinematic model.

where $\mathbf{u} = [v\ \omega]^{\mathrm{T}}$ is the control input vector that contains the linear velocity v and the angular velocity ω. The backward motion of the car-like robot is not considered in this study: that is, v is positive.

In this kinematic model, the motion of the car-like robot constrained by the following nonholonomic constraint is described as

$$\dot{x}\sin\theta - \dot{y}\cos\theta = 0 \tag{2}$$

That is, it is assumed that the car-like robot is reasonably slow such that the longitudinal traction and lateral force exerted on the tires do not exceed the maximum static friction between the tires and the floor. This is called a no-slip condition.

At the low speed of the car-like robot, the kinematic steering is determined from the Ackerman turning geometry as $\delta = b/R = b\kappa$ where δ is the equivalent steering angle for the front wheels, b the wheelbase, and R the turning radius. Due to the maximum value of the equivalent steering angle for the front wheels $\delta_{\max}$, the lower limit on the turning radius is given by $R_{\min}$.

Local information from the range sensor. To consider a realistic navigation algorithm, we assume that the car-like robot is equipped with a range sensor with the limited sensing range shown in Figure 2. The maximum sensing range for the range sensor is defined as $r_{\max}$ and the FOV of the range sensor is also limited by 2β. Even if the range sensor provides the set of discrete points with respect to the angle resolution of sensor, they are considered as the continuous line or curve if the distance between the two points is close. Thus, the detected obstacle boundary is modeled as a thin wall, which is represented by the bold curves in Figure 2. Let the set of endpoints of the

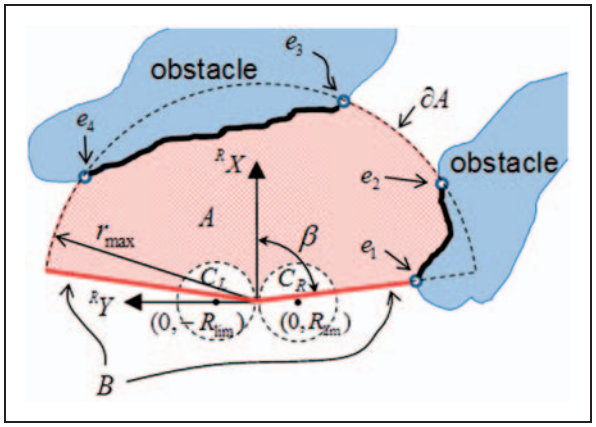

Figure 2. Local view from the range sensor of the robot.

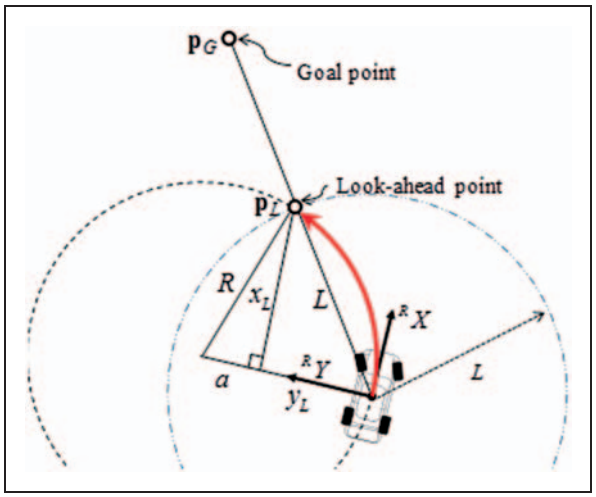

Figure 3. Geometry of the pure pursuit method.

obstacle boundary be the set E. For example, in Figure 2, $E = \{e_1, e_2, e_3, e_4\}$.

Let $A(\mathbf{q}, r_{\max}, \beta)$ be the area within the sensing distance and FOV. The arc boundary of A at radius $r_{\max}$ is defined as ∂A, and B is the union of the two bounding rays. Then, the local free space is

$$F_{free} = A - \sum_{i=1}^{n} WO_i \tag{3}$$

where WO_i is an ith obstacle and n the number of obstacles.

The robot cannot directly move inside the circles C_L and C_R shown in Figure 2 due to the minimum turning radius. Then, the local feasible space is defined as

$$F_{feas} = F_{free} - (C_L \cup C_R) \tag{4}$$

where C_L and C_R are the interior of the circles centered at $(0, -R_{\text{lim}})$, $(0, R_{\text{lim}})$, respectively. Thus, to insure that the collision-free motion of the car-like robot is guaranteed, the instantaneous motion of the robot should be planned inside of F_{feas} during the navigation.

Pure pursuit method for instantaneous robot motion. For each behavior of the motion-to-goal and wall-following, the proposed algorithm uses the pure pursuit method[12] to plan the local nonholonomic motion. By calculating the turning radius at every sample time, the motion constraints on the robot is satisfied during the navigation.

Figure 3 shows the geometry of the pure pursuit for the car-like robot. In Figure 3, an arc that joins the current position of the robot and look-ahead point $\mathbf{p}_L$ can be constructed. The chord length of this arc is the look-ahead distance L. Also, let the location of the look-ahead point be (x_L, y_L) with respect to the robot coordinate. Then, from Figure 3, ${x_L}^2 + {y_L}^2 = L^2$ and $R = a + y_L$ can be geometrically obtained where a satisfies ${x_L}^2 + a^2 = R^2$. With these equations, the turning radius is obtained as follows

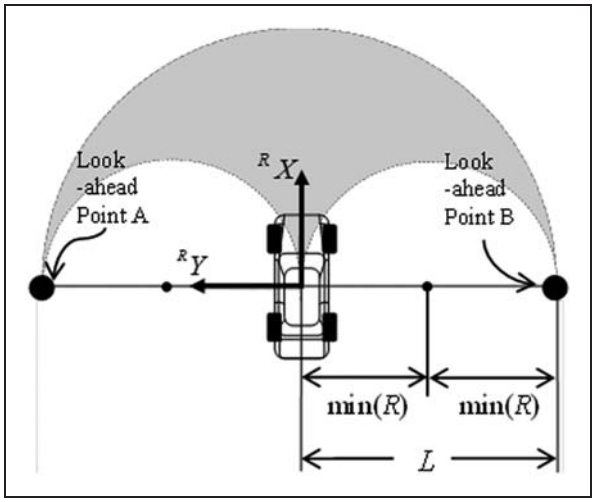

Figure 4. Set of feasible trajectory with pure pursuit method.

$$R = \frac{L^2}{2y_L} \tag{5}$$

Thus, we can see that the turning radius is proportional to the square of the look-ahead distance. With the goal point $\mathbf{p}_G$ shown in Figure 3, the turning radius is calculated using equation (5). This turning radius specifies the instantaneous motion of the car-like robot, and it can be shown that the robot reaches $\mathbf{p}_G$ if the previous steps are repeated.[11]

With the look-ahead distance L, the shaded region of Figure 4 represents the set of the feasible trajectory of the car-like robot under the pure pursuit method. As the instantaneous motion is generated using the pure pursuit method, the look-ahead point can be placed at any position on the circumference of the

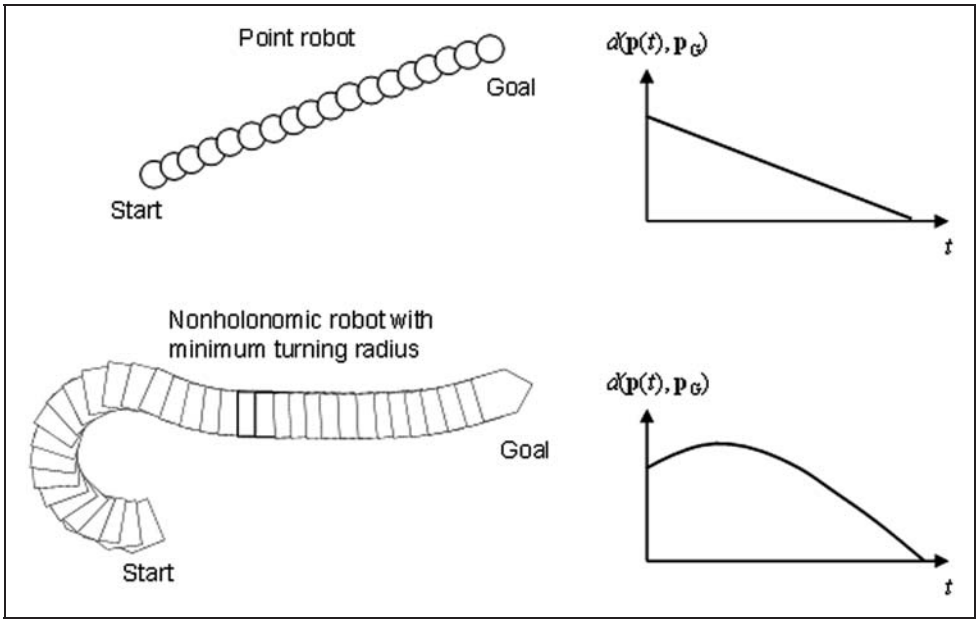

Figure 5. Comparison of the Euclidean distance to the goal between two robots: the point robot (upper) and the nonholonomic robot (lower).

circle of which the center is the robot position and the radius the look-ahead distance L.

If the orientation of the robot is exactly toward the instantaneous goal point, then the look-ahead point lies on the intersection of the X-axis of the robot coordinate and the circumference of the circle with radius L. As the orientation of the robot moves away from the instantaneous goal point, the look-ahead point moves toward the points A or B along the circumference of the circle with radius L. In Figure 4, the look-ahead points A and B are special cases where the look-ahead points are on the Y-axis of the robot coordinate. For those cases, the pure pursuit method calculates the minimum value of the turning radius $\min(R)$ as $R_{\min}$. Then, the look-ahead distance L and the minimum turning radius have the relationship shown in Figure 4

$$L = 2\min(R) = 2R_{\min} \text{ or } R_{\min} = \frac{L}{2} \tag{6}$$

Thus, the look-ahead distance L needs to be selected such that it satisfies $R_{\min} > R_{\lim}$, where $R_{\lim}$ is the lower limit on the turning radius determined by equation (5).

Motion-to-goal behavior

This section describes the details of the motion-to-goal behavior for the car-like robots. Let the function $d(\mathbf{p}, \mathbf{p}_G)$ calculate the Euclidean distance of a point $\mathbf{p}$ from $\mathbf{p}_G$, where $\mathbf{p} = [x \ y]^{\mathrm{T}}$ is the coordinate of the robot. During the motion-to-goal behavior, the robot moves toward the goal point while $d(\mathbf{p}, \mathbf{p}_G)$ decreases. The robot finds the local path called the instantaneous nonholonomic path using the pure pursuit method.

If this local path to the goal is collision free within F_{feas} given by equation (4), then the robot moves directly to the goal (direct motion-to-goal) using the nonholonomic path to the local goal. Thus, the look-ahead point locates on the line connecting the start point of the direct motion-to-goal and the goal point. Otherwise, if that path intersects with the obstacle within the sensing range, the robot should avoid the obstacle while the distance to the goal decreases. In this case of the slide motion-to-goal behavior, the look-ahead point locates on the line connecting the current position and the endpoint of the obstacle boundary. This behavior is termed as the slide motion-to-goal. The motion-to-goal behavior continues until $d(\mathbf{p}, \mathbf{p}_G)$ begins to increase, then the robot meets the local minima point. This point then switches to the wall-following behavior.

So far, these two modes are similar to those of the Tangent Bug algorithm, but since we are considering a nonholonomic robot, the following problem needs to be considered. Depending on the orientation of the car-like robot at the beginning of the motion-to-goal, $d(\mathbf{p}, \mathbf{p}_G)$ can increase even when there is no obstacle blocking the robots path. Figure 5 shows examples of the motion-to-goal and the distance to goal for the two different robots: the upper one is the holonomic point robot and the lower one is the car-like robot with a minimum turning radius. For a holonomic point robot, $d(\mathbf{p}, \mathbf{p}_G)$ continuously decreases during the motion-to-goal behavior, while $d(\mathbf{p}, \mathbf{p}_G)$ increases initially for the car-like robot. Regarding this motion as a switching point to the wall-following behavior can cause problems, so this article introduces

conditionally determined motion behavior. If the robot does not meet the obstacle until $d(\mathbf{p}, \mathbf{p}_G)$ becomes smaller than the recorded minimum distance to the goal point, the local path segment is regarded as the motion-to-goal segment. Otherwise, the robot should avoid obstacles with the wall-following behavior.

Blocking obstacle with the instantaneous nonholonomic path. For a holonomic robot, the blocking obstacle intersects with the holonomic path to the goal, the straight line connecting the robot and the goal point. However, the car-like robot with the minimum turning radius cannot move directly to the goal which guarantees the local optimal direction. Even if the holonomic path to the goal point is collision free, the nonholonomic path generated by implementing the pure pursuit may intersect with the obstacle. Thus, the blocking obstacle for the car-like robot should be defined using the nonholonomic constraint.

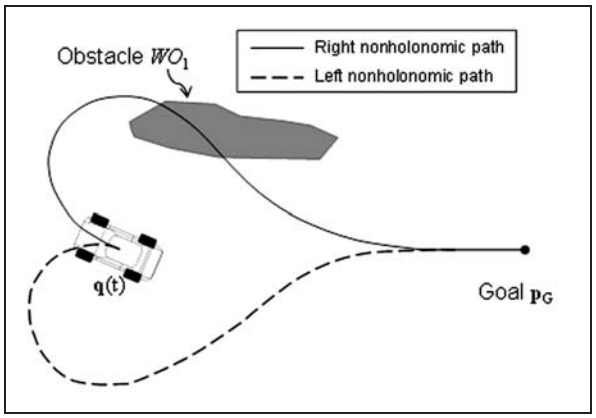

Figure 6. Example of the blocking obstacle for a car-like robot.

The left and right nonholonomic paths are defined as the paths to the goal point starting with the right and left turns of the robot, respectively. Each path is generated by applying the pure pursuit method to the kinematic model of the robot, and the sequence of points that correspond with the nonholonomic path to the goal can be obtained. That is, the robot has the two choices shown in Figure 6, the left nonholonomic path (dotted line) or the right nonholonomic path (solid line). The algorithm selects the one that does not intersect with the obstacle within the sensing range. If neither intersects the obstacles, then the shorter one is chosen, termed the instance nonholonomic path. Otherwise, if there is no collision-free path, the robot cannot move directly to the goal point due to the blocking obstacle. In this case, the blocking obstacle requires the termination of the direct motion-to-goal and the beginning of the slide motion-to-goal to avoid the obstacle. For the example shown in Figure 6, the robot can select the left nonholonomic path even when the length of this path is longer than the other path.

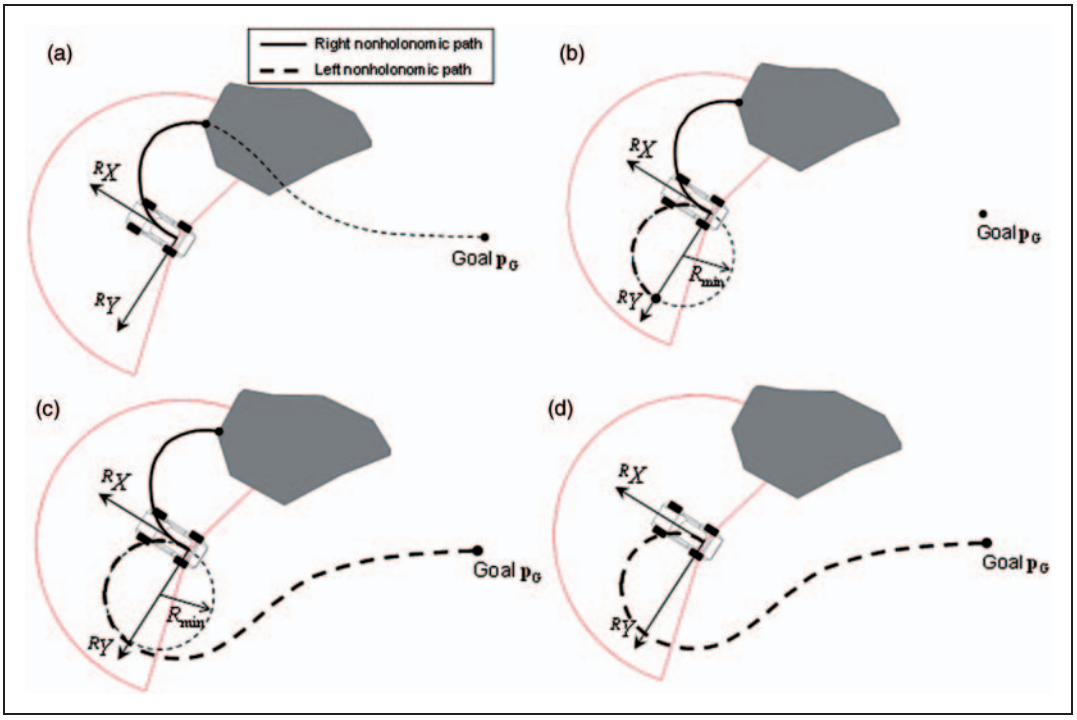

Figure 7. Example of processes for finding the instant nonholonomic path: (a) extend the right nonholonomic path until it collides with the obstacle within the sensing range; (b) plan the circular path with radius R_{min} from $\mathbf{q}$ until ^RX reaches the opposite direction of $\mathbf{p}_G - \mathbf{p}$; (c) extend the left nonholonomic path from the point obtained in (b) to $\mathbf{p}_G$; and (d) determine the instant nonholonomic path to $\mathbf{p}_G$.

Let us introduce the left and right nonholonomic paths to the goal defined as $LNP(s) = [x_{LNP}(s)\ y_{LNP}(s)\theta_{LNP}(s)]^{\mathrm{T}}$ and $RNP(s) = [x_{RNP}(s)\ y_{RNP}(s)\ \theta_{RNP}(s)]^{\mathrm{T}}$, respectively, where s is the time step. With a given step size Δs, the paths are expressed as the set of discrete points of the robot configuration.

As the first step for finding the instantaneous nonholonomic path, from the kinematic model of the robot given in equation (1), the turning radius calculated by equation (5) is used to calculate the each $LNP(s)$ or $RNP(s)$ with the step size Δs (see Algorithm 2). If this path does not meet the obstacle within the sensing range, then we conclude that this path is the instantaneous nonholonomic path to the goal point $\mathbf{p}_G$. For checking the collision between the nonholonomic path and the obstacle boundary, the obstacle distances from the point to the each obstacle point OP_k within the sensing range, where $k = 1, 2, 3, \ldots, j$ are calculated. However, as shown in Figure 7(a), the nonholonomic path may meet the obstacle. Then, a second step is required to find the other nonholonomic path. In this case, we build the circular path with $R_{\min}$ from the initial configuration to the place where the orientation of the robot reaches the opposite direction of $\mathbf{p}_G - \mathbf{p}$ (see Algorithm 1). Then, for the third step, the nonholonomic path is generated toward the same as the first step as shown in Figure 7(c). In the fourth step, if this path is collision free within the sensing range, the algorithm determines the instantaneous nonholonomic paths (see Algorithms 3 and 4). In the example of Figure 7, the left nonholonomic path is determined as the instantaneous nonholonomic path.

The instantaneous nonholonomic path does not exist if both the paths intersect with the obstacle within the sensing range. Thus, as described in Algorithm 5, the algorithm concludes that a collision will result and detects the blocking obstacle. Of course, if only one of the paths is a collision-free path, then the algorithm returns the collision-free one. Otherwise, if both paths are collision free, the shorter one is selected as the nonholonomic path for the robot.

Motion-to-goal algorithm for a car-like robot. As we already discussed, due to the motion constraint, the distance to the goal point can increase even if there is no obstacle. We first examine the case when the robot satisfies $(\mathbf{p}_G - \mathbf{p}) \cdot {}^{R}X \geqslant 0$ at the beginning of direct motion-to-goal so that the distance to goal decreases.

According to 'Blocking obstacle with the instantaneous nonholonomic path' section, the algorithm results in an instantaneous nonholonomic path to the goal or the existence of the blocking obstacle during the direct motion-to-goal behavior. The motion-to-goal algorithm executes the direct motion-to-goal behavior for the former and the slide motion-to-goal behavior for the latter.

Algorithm 1 Generate circular path to change the direction of robot opposite

Input: A local view from the range sensor and $\mathbf{q}$, $\mathbf{p}_G$, L
Output: Collision status or circular path $NP(s)$ from $\mathbf{q}$ to $\mathbf{p}_G$

$NP(s) \leftarrow \mathbf{q}$
$\mathbf{q}_{NP}(s) \leftarrow \mathbf{q}$
while $\theta_{NP}(s)\mathbf{q}_{NP}(s)$ reaches to the opposite direction of $(\mathbf{p}_G - \mathbf{p})$
$R \leftarrow \mathrm{R}_{\min}$
$\mathbf{u} \leftarrow [v\ v/R]^{\mathrm{T}}$
$\mathbf{q}_{NP} \leftarrow \mathrm{model}(\mathbf{u}, \Delta s)$
if $\mathbf{q}_{NP}$ intersect with the obstacle boundary, $d(\mathbf{q}_{NP}(s), \mathrm{OP}_j) < d_{safe}$ where $j = 1, 2, \ldots, k$
return collision
break
else
$NP(s) \leftarrow \mathbf{q}_{NP}$
end if
$s \leftarrow s + \Delta s$
end while
return $NP(s)$

Algorithm 2 Generate the nonholonomic path using pure pursuit

Input: A local view from the range sensor and $\mathbf{q}$, $\mathbf{p}_G$, L
Output: Collision status or circular path $NP(s)$ from $\mathbf{q}$ to $\mathbf{p}_G$

$NP(s) \leftarrow \mathbf{q}$
$\mathbf{q}_{NP}(s) \leftarrow \mathbf{q}$
while $NP(s) \neq \mathbf{q}_G$
$R \leftarrow \mathrm{purepursuit}(\mathbf{q}_{NP}(s), \mathbf{p}_G, L)$
$\mathbf{u} \leftarrow [v\ v/R]^{\mathrm{T}}$
$\mathbf{q}_{NP} \leftarrow \mathrm{model}(\mathbf{u}, \Delta s)$
if $\mathbf{q}_{NP}$ intersect with the obstacle boundary, $d(\mathbf{q}_{NP}(s), \mathrm{OP}_j) < d_{safe}$ where $j = 1, 2, \ldots, k$
return collision
break
else
$NP(s) \leftarrow \mathbf{q}_{NP}$
end if
$s \leftarrow s + \Delta s$
end while
return $NP(s)$

First, we investigate the details of the direct motion-to-goal behavior. This behavior is required when there are no blocking obstacles defined in 'Blocking obstacle with the instantaneous

Algorithm 3 Build the left nonholonomic path

Input: A local view from the range sensor and $\mathbf{q}$, $\mathbf{p}_G$, L
Output: Collision status or the left nonholonomic path $LNP(s)$ from $\mathbf{q}$ to $\mathbf{p}_G$

$s \leftarrow 0$
$\theta_{\mathbf{PPG}} \leftarrow$ angle between ${}^{R}X$ and $(\mathbf{p}_G - \mathbf{p})$
if $\theta < \theta_{\mathbf{PPG}}$
$LNP(s) \leftarrow$ Generate the nonholonomic path ($\mathbf{q}$, $\mathbf{p}_G$, L)
if $LNP(s)$ collides with the obstacle
return LNP_collision
end if
return $LNP(s)$
else if $\theta > \theta_{\mathbf{PPG}}$
  $LNP(s) \leftarrow$ Generate circular path ($\mathbf{q}$, $\mathbf{p}_G$, L)
  if $LNP(s)$ collides with the obstacle
  return LNP_collision
  else
  $LNP(s) \leftarrow$ Generate the nonholonomic path ($\mathbf{q}$, $\mathbf{p}_G$, L)
  if $LNP(s)$ collides with the obstacle
  return LNP_collision
  end if
end if
return $LNP(s)$
end if

Algorithm 4 Build the right nonholonomic path

Input: A local view from the range sensor and $\mathbf{q}$, $\mathbf{p}_G$, L.
Output: Collision status or the right nonholonomic path $LNP(s)$ from $\mathbf{p}$ to $\mathbf{p}_G$.

$s \leftarrow 0$
$\theta_{\mathbf{PPG}} \leftarrow$ angle between ${}^{R}X$ and $(\mathbf{p}_G - \mathbf{p})$
if $\theta > \theta_{\mathbf{PPG}}$
$RNP(s) \leftarrow$ Generate the nonholonomic path ($\mathbf{q}$, $\mathbf{p}_G$, L)
if $RNP(s)$ collides with the obstacle
return RNP_collision
end if
return $RNP(s)$
else if $\theta < \theta_{\mathbf{PPG}}$
  $RNP(s) \leftarrow$ Generate circular path ($\mathbf{q}$, $\mathbf{p}_G$, L)
  if $RNP(s)$ collides with the obstacle
  return RNP_collision
  else
  $RNP(s) \leftarrow$ Generate the nonholonomic path ($\mathbf{q}$, $\mathbf{p}_G$, L)
  if $RNP(s)$ collides with the obstacle
  return RNP_collision
  end if
end if
return $RNP(s)$
end if

nonholonomic path' section, and the robot just needs to approach to the instantaneous goal point directly. Let S_i be a point where the robot starts the ith direct motion-to-goal. During this behavior, as shown in Figure 8, the look-ahead point $\mathbf{p}_L$ lies on the line between S_i and $\mathbf{p}_G$ where it satisfies $d(\mathbf{p}, \mathbf{p}_L) = L$ and $(\mathbf{p}_L - \mathbf{p}) \cdot {}^{R}X > 0$. Then, the instantaneous motion for the direct motion-to-goal is determined as the turning radius R using equation (5). Therefore, during this direct motion-to-goal behavior, the robot just moves along the nonholonomic path from S_i to $\mathbf{p}_G$ until it detects the blocking obstacle.

When the blocking obstacle is detected, the direct motion-to-goal behavior is switched to the slide motion-to-goal. This behavior finds the focus point from the sensed endpoints and the instantaneous motion for the robot is determined. Thus, these processes are represented as follows: to guarantee the shortest path to goal point, the focus point F is determined as $F = e_{\min}$ with the following equation.

$$e_{\min} = \arg\min_{e_i \in E} \{ d(\mathbf{p}, e_i) + d(e_i, \mathbf{p}_G) \} \tag{7}$$

To generate the collision-free motion with d_{safe} where d_{safe} is the safe distance between the robot and the obstacle boundary, F' is defined to be a point d_{safe}

Algorithm 5 Determine the nonholonomic path for robot or the blocking obstacle

Input: A local view from the range sensor and $\mathbf{q}$, $\mathbf{p}_G$, L
Output: Blocking obstacle status or the nonholonomic path for the robot status from $\mathbf{q}$ to $\mathbf{p}_G$

$LNP(s) \leftarrow$ Build the left nonholonomic path($\mathbf{q}$, $\mathbf{p}_G$, L)
$RNP(s) \leftarrow$ Build the right nonholonomic path($\mathbf{q}$, $\mathbf{p}_G$, L)
if $LNP(s)$ and $RNP(s)$ intersect with the obstacle boundary
return blocking obstacle
else if $LNP(s)$ returns collision, but $RNP(s)$ returns the path
return $RNP(s)$
else if $LNP(s)$ the path, but $RNP(s)$ returns collision
return $LNP(s)$
else
  $NP(s) \leftarrow$ select the path which has the shorter one of $LNP(s)$ and $RNP(s)$
  return $NP(s)$
end if

away from $F = e_{\min}$ along the surface normal vector of the obstacle boundary at $e_{\min}$. Then, the focus point becomes F' and the look-ahead point $\mathbf{p}_L$ lies on the line between $\mathbf{p}$ and F', as shown in Figure 9. It shows the two cases for the slide motion-to-goal behavior when F lies on the boundary of A, ∂A (left) and the interior of A, int(A) (right).

The robot must avoid the obstacle boundary with the fixed travel direction during the slide motion-to-goal behavior to prevent the sudden change of the direction of rotation which causes the oscillation or divergence of the path. The travel direction is determined at the beginning of the slide motion-to-goal behavior. To prevent the sudden change of focus point location, the algorithm records the previous location of F, e_i for a single obstacle boundary. Thus, the endpoint which is close to the previous one on the single obstacle boundary is selected. Especially, if $e_{\min}$ lies on B, $e_{\min} \in B$, as shown in Figure 10. When the robot slides the obstacle boundary, the location of F is determined from the following equation.

$$e_{a\min} = \arg\min_{e_i \in E} \cos^{-1} \frac{{}^R X \cdot \mathbf{t}_{ei}}{|{}^R X||\mathbf{t}_{ei}|} \tag{8}$$

where $\mathbf{t}_{ei}$ is the tangent vector at e_i and e_i the endpoint lying on the followed obstacle boundary. As the set of obstacle points is obtained from the range sensor, $\mathbf{t}_{ei}$ is obtained using the two obstacle points that e_i and the obstacle point lie on the obstacle boundary and closest to e_i.

The slide motion-to-goal behavior is terminated and the direct motion-to-goal begins if the instantaneous nonholonomic path to $\mathbf{p}_G$ defined in 'Blocking obstacle with the instantaneous nonholonomic path' section is found. Otherwise, it is switched to

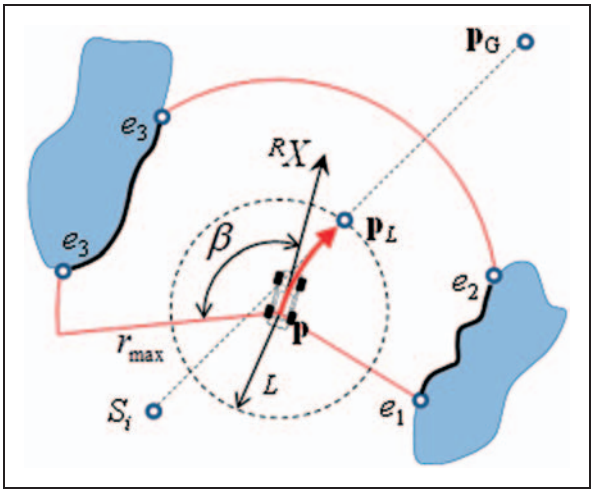

Figure 8. Example of the direct motion-to-goal when the robot satisfies $(\mathbf{p}_G - \mathbf{p}) \cdot {}^R X \geqslant 0$ at S_i.

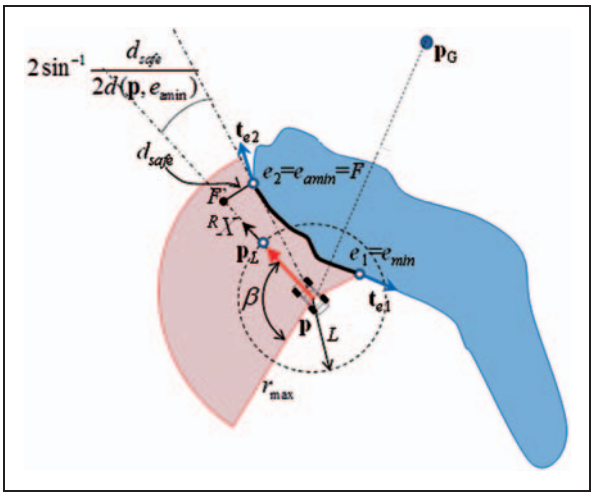

Figure 10. Example of the slide motion-to-goal when $e_{\min} \in B$.

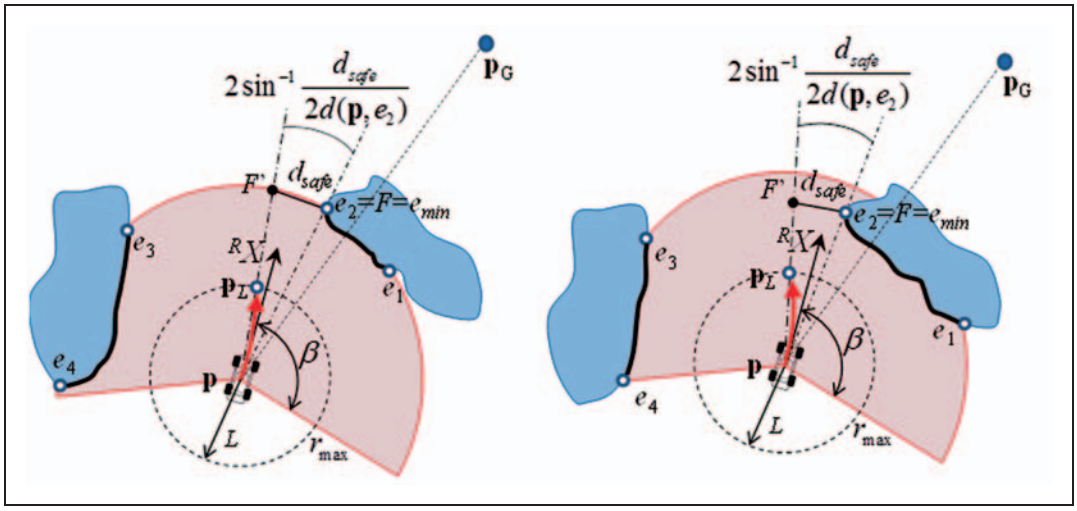

Figure 9. Example of the slide motion-to-goal when $F \in \partial A$ (left) and $F \in \text{int}(A)$ (right) for $F = e_{\min}$.

the wall-following behavior if the distance to goal $d(\mathbf{p}, \mathbf{p}_G)$ begins to increase. In this case, the ith local minimum point is determined as $\mathrm{M}_i = \mathbf{p}(t)$ and the minimum distance to goal is recorded as $d_{\min} = d(\mathrm{M}_i, \mathbf{p}_G)$. This is the leaving condition from the motion-to-goal to wall-following. The local minimum point plays an important role as it terminates the motion-to-goal behavior and starts the wall-following behavior.

Wall-following

The robot follows the obstacle boundary with the fixed travel direction determined at the slide motion-to-goal behavior. If e_{amin} given in equation (8) lies on the right (or left) side of the robot, the direction becomes the counterclockwise (or clockwise). Then, as shown in Figure 11, the robot moves around the obstacle boundary by following the look-ahead point $\mathbf{p}_L$ with the focus point F', where F' is defined to be the point d_{safe} away from the focus point $F = e_{\mathrm{amin}}$ along the surface normal vector of the obstacle boundary at e_{amin}. Thus, during this behavior, the focus point F lies on the arc boundary of the sensing area ∂A.

As the robot detects the local minima M_i at the end of the motion-to-goal behavior, we have already obtained the minimum distance to the goal $d_{\min}(\mathbf{p}_G)$. If the focus point F' is found where the distance from F' to $\mathbf{p}_G$ is smaller than $d_{\min}(\mathbf{p}_G)$, then the robot leaves the obstacle boundary. Then, the robot moves toward $\mathbf{p}_G$ until it reaches the leaving point Z that satisfies $d(\mathrm{Z}, \mathbf{p}_G) < d_{\min}(\mathbf{p}_G)$. When the robot meets the leaving point, it resumes the motion-to-goal behavior.

Conditionally determined motion behavior. A special treatment is required when $(\mathbf{p}_G - \mathbf{p}) \cdot {}^R X < 0$ at the beginning of motion-to-goal behavior. Even if the distance to the goal increases due to the minimum turning radius of the robot, the behavior should not switch to the wall-following but remain as the motion-to-goal if the robot does not meet the blocking obstacle. However, we cannot assume that the blocking obstacle on the robot's path is due to the locally available sensor information. To cope with this problem, we introduce the conditionally determined motion behavior where the path segment is changed into the wall-following segment or the motion-to-goal segment after the robot follows the nonholonomic path described in 'Blocking obstacle with the instantaneous nonholonomic path' section.

The algorithm of conditionally determined motion behavior is divided into the two cases. First, the range sensor with a large sensing range is considered so that the robot can see the both paths to points $\mathrm{P_R}$ or $\mathrm{P_L}$, where $\mathrm{P_R}$ and $\mathrm{P_L}$ are the intersection points between the nonholonomic paths and the circle centered at $\mathbf{p}_G$ with the radius $d(\mathbf{p}, \mathbf{p}_G)$. If the algorithm finds the nonholonomic path to $\mathrm{P_R}$ or $\mathrm{P_L}$ to be like the one shown in Figure 12 (left), the robot follows the nonholonomic path shown in Figure 12 (right) until

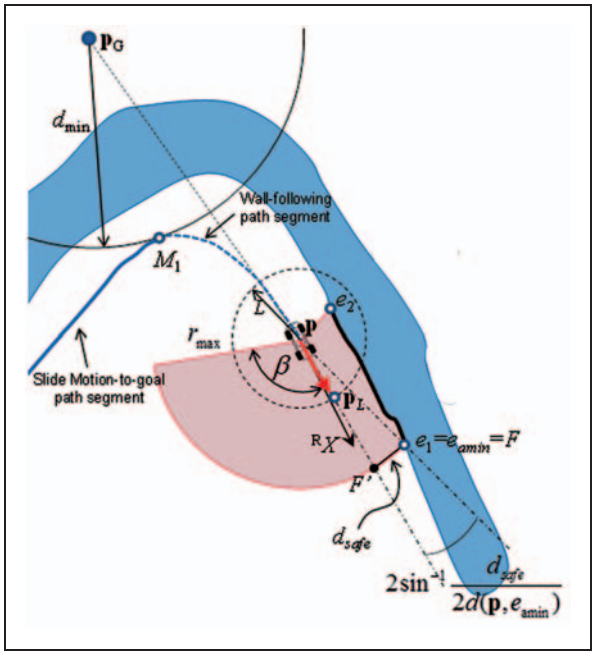

Figure 11. Wall-following behavior of the car-like robot.

$d(\mathbf{p}, \mathbf{p}_G) < d(\mathrm{P_L}, \mathbf{p}_G,)$ or $d(\mathbf{p}, \mathbf{p}_G) < d(\mathrm{P_R}, \mathbf{p}_G,)$ is satisfied. Then, the path segment behavior becomes the motion-to-goal path segment. Otherwise, if the blocking obstacle is detected where both the left and right nonholonomic paths are not the collision-free paths, the conditionally determined motion segment becomes the wall-following segment according to 'Wall-following' section.

Second, the robot cannot see the nonholonomic paths from $\mathbf{p}$ to $\mathrm{P_R}$ or $\mathrm{P_L}$ shown in Figure 13. If the robot can reach $\mathrm{P_R}$ or $\mathrm{P_L}$ without meeting the obstacle, then the followed path from the conditionally determined motion behavior becomes the motion-to-goal segment. However, the robot may meet the unexpected obstacle on the path while the it moves along the nonholonomic path. For this case, $d_{\min}(\mathbf{p}_G)$ and $d_{\max}(\mathbf{p}_G)$ are defined as the minimum and maximum distances to the goal during the conditionally determined motion behavior, respectively.

The robot follows the wall using the focus point F' for the wall-following mode explained in 'Wall-following' section, and continuously updates the distance $d_{\max}(\mathbf{p}_G)$. If the distance to the goal $d(\mathbf{p}, \mathbf{p}_G)$ begins to decrease, then $d_{\max}(\mathbf{p}_G)$ is determined as the local maximum point, MA_i. While the algorithm continues the wall-following segment, the focus point F' for the motion-to-goal behavior is used for sliding and directly moving toward the goal point until the robot reaches another local minima point or it satisfies $d(\mathbf{p}, \mathbf{p}_G) < d_{\min}(\mathbf{p}_G)$. In the case of local minima detection, the robot performs the wall-following again until it detects another MA_i.

To leave the blocking obstacle, we consider the two sub-cases, as shown in Figure 13. For the first subcase, the robot leaves at the point where $d(\mathbf{p}, \mathbf{p}_G) < d_{\min}(\mathbf{p}_G)$ is satisfied. Thus, the behavior is

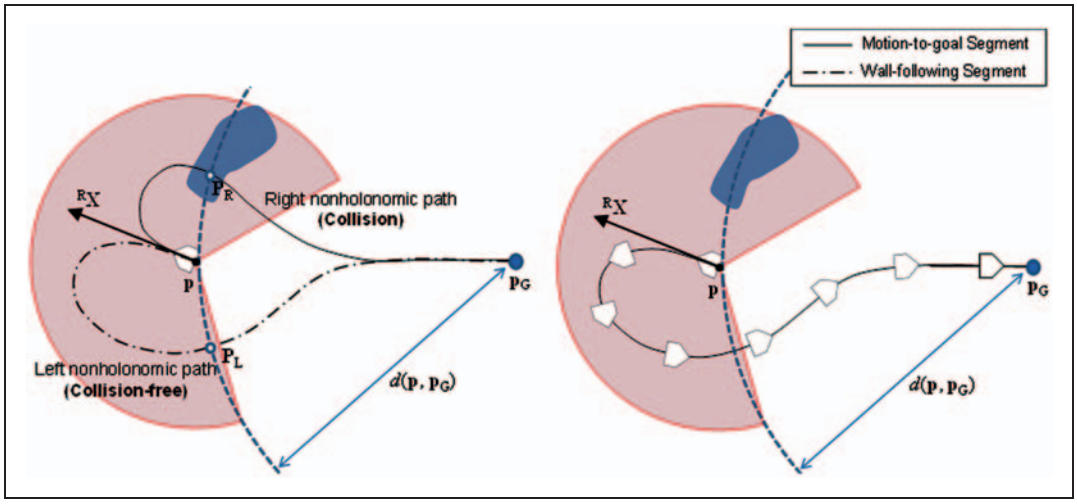

Figure 12. First case: the robot can see the both paths to points P_R and P_L. After the nonholonomic path is selected (left), the robot follows the nonholonomic path (right).

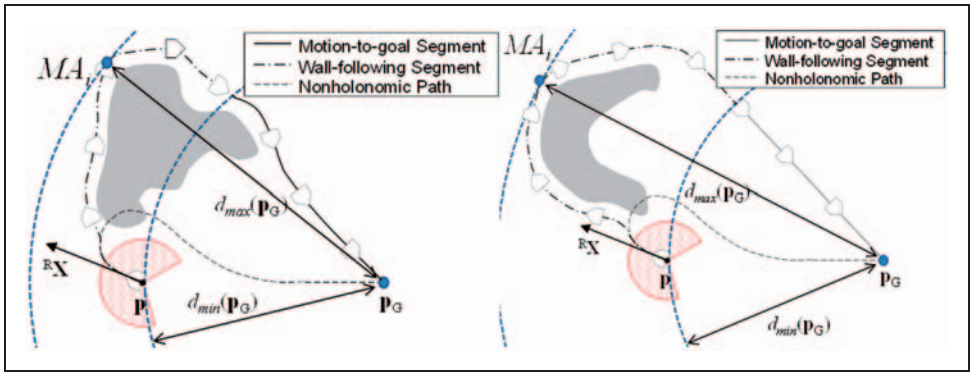

Figure 13. Second case: the robot cannot see the both paths to points P_R and P_L. The some part of blocking obstacle is included within the circle radius $d_{\min}(\mathbf{p}_G)$ (left) or not (right) at the switching point to the motion-to-goal behavior.

switched from the wall-following to the motion-to-goal. Figure 13 (left) shows the example of this case. At this leaving point, the robot moves with the slide motion-to-goal behavior, and after that, the direct motion-to-goal behavior. For the second sub-case shown in Figure 13 (right), the robot leaves the blocking obstacle before it satisfies $d(\mathbf{p}, \mathbf{p}_G) < d_{\min}(\mathbf{p}_G)$. After the robot meets the local maxima point MA_i, the behavior is determined from the motion-to-goal; thus, the robot moves directly to the goal point if the blocking obstacle disappears. Even so, the resultant path until the robot $d(\mathbf{p}, \mathbf{p}_G) < d_{\min}(\mathbf{p}_G)$ becomes the wall-following segment.

Analysis of the algorithm

In this section, we examine the two issues on the navigation algorithm: the correctness and the completeness. First, the correctness is defined as the property where the local motion of the car-like robot is collision free. We describe the conditions for correctness in order to prevent the backward motion of the car-like robot being required to avoid the obstacle. In addition, the instantaneous turning radius should navigate the car-like robot within the known collision-free region from the range sensor. Second, the correctness assures that the path to the goal point can be found. That is, the convergence to the goal is provided by investigating the finite length of the path for each behavior mode.

Conditions for correctness

We assume that the look-ahead distance is given by L and the safe distance d_{safe} can be ignored. As shown in Figure 14, we consider the case $0 < \beta < \pi/2$ which is half of the sensing angle. With the notion of the pure pursuit, the focus point which corresponds to the local goal exists on the boundary of the circular arc from A to A'. Thus, the pure pursuit method cannot calculate the turning radius smaller than $R'_{\min}$ in Figure 14 even though $R'_{\min}$ is larger than $R_{\min}$ obtained from equation (6). This means that the small sensing angle increases the size of minimum turning radius.

Otherwise, if β is larger than $\pi/2$, the robot has the minimum turning radius $R_{\min} = L/2$ from equation (6). Therefore, the minimum turning radius of the robot is calculated as the function of β as

$$R_{\min}(\beta) = \begin{cases} \dfrac{L}{2\sin\beta} & \text{if } 0 < \beta < \dfrac{\pi}{2} \\ \dfrac{L}{2} & \text{otherwise} \end{cases} \tag{9}$$

To calculate the feasible size of turning radius, the minimum turning radius $R_{\min}(\beta)$ should be larger than $R_{\lim}$ given in equation (6). Then, using equations (6) and (9), the condition for selecting the look-ahead distance L is required so that $R_{\min}(\beta) > R_{\lim}$ is satisfied for $L > 2\,(l_f + l_r)/\delta_{\max}$.

Let us describe how the length of the maximum sensing range $r_{\max}$ affects the correctness of the algorithm. The case where the robot cannot come out from the obstacle boundary without the backward motion should be avoided because we consider the car-like robot to only be moving forward. Let us assume that $r_{\max} < 2R_{\lim}$ than the collision-free path does not exist due to the minimum turning radius of the robot. Then, we consider that the maximum sensing distance satisfies $r_{\max} > 2R_{\lim}$ so that the only forward motion of car-like robot is required during the navigation. Let the focus point F', which is the local goal, be required to exist within the local feasible space F_{feas} defined in equation (4). With these constraints, we consider the instantaneous robot motion with two cases shown in Figure 15. The first case is shown in Figure 15 (left) when the maximum sensing range satisfies $r_{\max} > 2R_{\min}(\beta) > 2R_{\lim}$. The instantaneous motion is determined based on the pure pursuit method, thus the turning radius is obtained with the look-ahead point $\mathbf{p}_L$. This look-ahead point also exists on the boundary of circular arc from A to A'. For the second case where the maximum sensing range satisfies $2R_{\min}(\beta) > r_{\max} > 2R_{\lim}$, the robot's look-ahead point $\mathbf{p}_L$ corresponds to the focus point F'. By changing the location of $\mathbf{p}_L$ from the boundary of circular arc to F', the robot can reach the focus point.

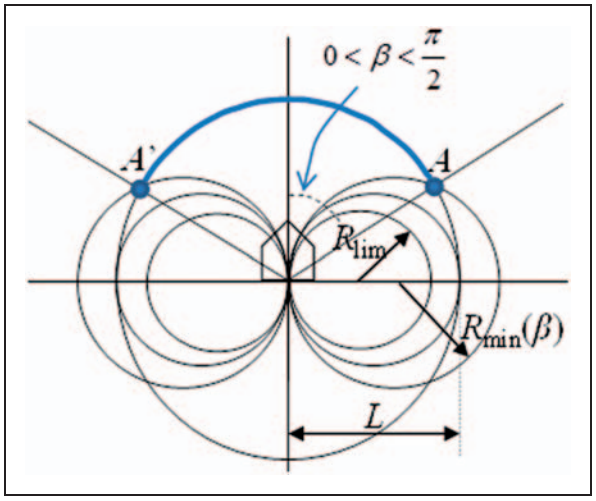

Figure 14. Increase of minimum turning radius when $0 < \beta < \pi/2$.

Proof of the completeness

As the navigation algorithm uses the notions from the Bug family algorithms, the proof of completeness is analogous to the Tangent Bug or Wedge Bug algorithm where the robot is assumed to be equipped with a finite-range sensor. Thus, this section shows the proof about the finite length of each behavior modes to be different because of the temporary

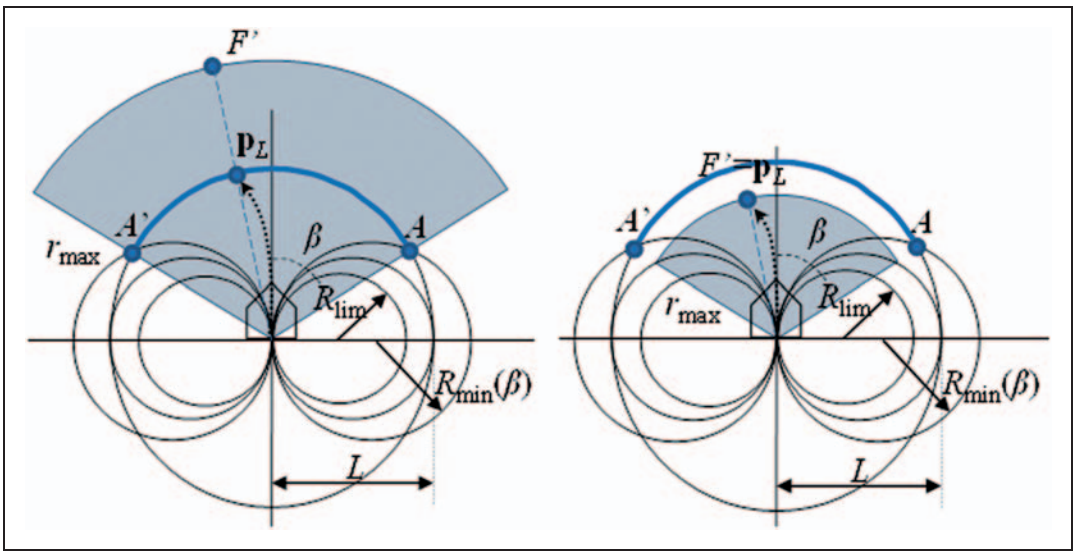

Figure 15. Instant robot motion when $r_{\max} > 2R_{\min}(\beta) > 2R_{\lim}$ (left) and $2R_{\min}(\beta) > r_{\max} > 2R_{\lim}$ (right).

nonholonomic behavior. Define the point S_i to be the ith point where the robot starts the direct motion-to-goal, P_i the ith point where the robot first senses the blocking obstacle and starts the slide motion-to-goal, M_i the ith point where the local minimum point is determined (the start point of the wall-following behavior), and Z_i the ith point where the robot leaves the obstacle.

Lemma 1. The distance to goal decreases each wall-following segment and between the wall-following segments.

Proof. During the wall-following behavior, the robot circumnavigates the obstacle boundary with a fixed direction until it reaches the leaving point where $d(\mathbf{p}, \mathbf{p}_G) < d_{\min}$ or the algorithm detects the loop. Thus, according to the switching condition, $d(Z_i, \mathbf{p}_G) < d(M_i, \mathbf{p}_G) = d_{\min}$ is guaranteed.

Lemma 2. The distance to goal decreases between the two motion-to-goal segments.

Proof. We consider the two cases for proving the convergence to goal between the two motion-to-goal segments. For a first case, if $(\mathbf{p}_G - \mathbf{p}) \cdot {}^R X >= 0$ at S_i, the conditionally determined motion behavior in 'Conditionally determined motion behavior' section is executed. The distance to goal decreases at the beginning of this behavior in this case. If the robot does not meet the blocking obstacle, the convergence of pure pursuit method guarantees that $d(\mathbf{p}, \mathbf{p}_G)$ becomes zero.[11] Otherwise, if the blocking obstacles appear during the motion-to-goal behavior, the behavior mode is switched to the wall-following behavior by determining the local minimum point M_i. Thus, this case guarantees that $d(S_i, \mathbf{p}_G) > d(M_i, \mathbf{p}_G)$.

For a second case, if $(\mathbf{p}_G - \mathbf{p}) \cdot {}^R X < 0$ at S_i, the conditionally determined motion behavior in 'Conditionally determined motion behavior' section is considered. Depending on the value of $r_{\max}$ and β for the range sensor, this behavior is divided into two cases. In the first case with a large sensing range, the algorithm determines whether the robot can approach to the point P_L or P_R (Figure 12) where it satisfies $d(\mathbf{p}, \mathbf{p}_G) < d(S_i, \mathbf{p}_G)$. Otherwise, as wall-following starts instead of the motion-to-goal, the proof of the finite-length path is guaranteed from Lemma 1. Also, in the second case with a small sensing range, the robot starts with the wall-following. During this behavior, the robot repeats the motion-to-goal and wall-following behaviors while it is determining the local maximum point MA_i and the local minimum point M_i. Thus, each path segment guarantees that the distance to goal decreases at the end of this behavior.

Experimental results

The proposed nonholonomic navigation algorithm was tested through several experiments. As shown in Figure 16, a differential drive mobile robot named PowerBot developed by ActiveMedia Robotics was used for testing the proposed approach. PowerBot is designed for educational purposes, and it is equipped with ultrasonic sensors, the range sensor. The low-level control like a wheel speed control is done with the on-board controller of the robot. The proposed navigation algorithm works on a laptop computer on the robot or the wireless communication enables the operation of the robot without including the laptop. For the experiments, the environment is made of the series of panels, as shown in Figure 17, and the size of each panel is $600 \times 400 \times 20\,\text{mm}^3$.

For all experiments, the linear velocity of the robot was fixed as 0.6 m/s, with a safe distance at 0.7 m. The measurable distance of the range sensor is about 9 m; however, we ignore the measured

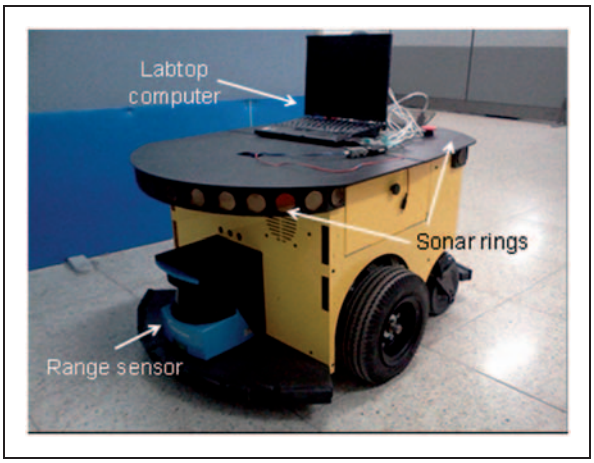

Figure 16. Mobile robot equipped with the laptop computer, range sensor, and sonar sensors.

Figure 17. Environment made of the series of panel.

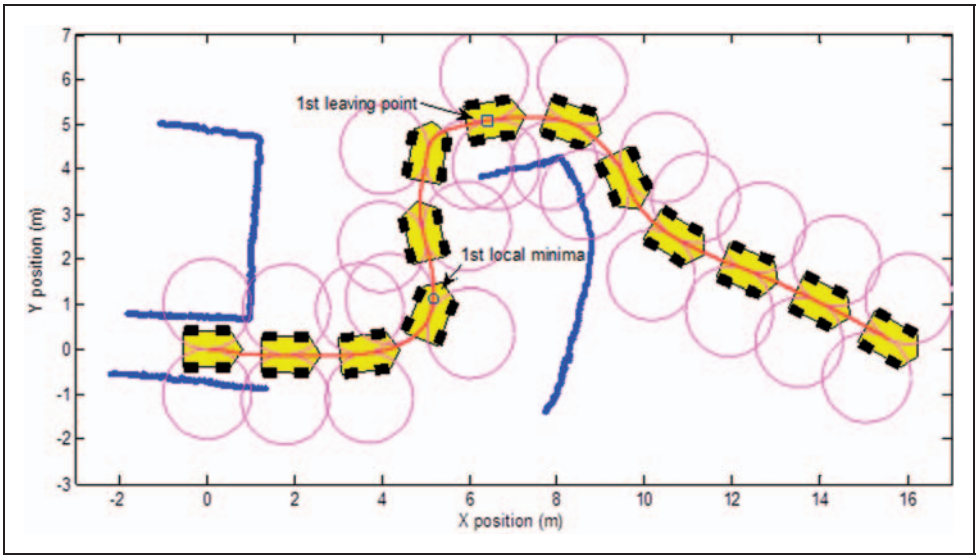

Figure 18. Experimental results in a simple environment when the initial orientation of the robot is zero, $\theta(0) = 0$.

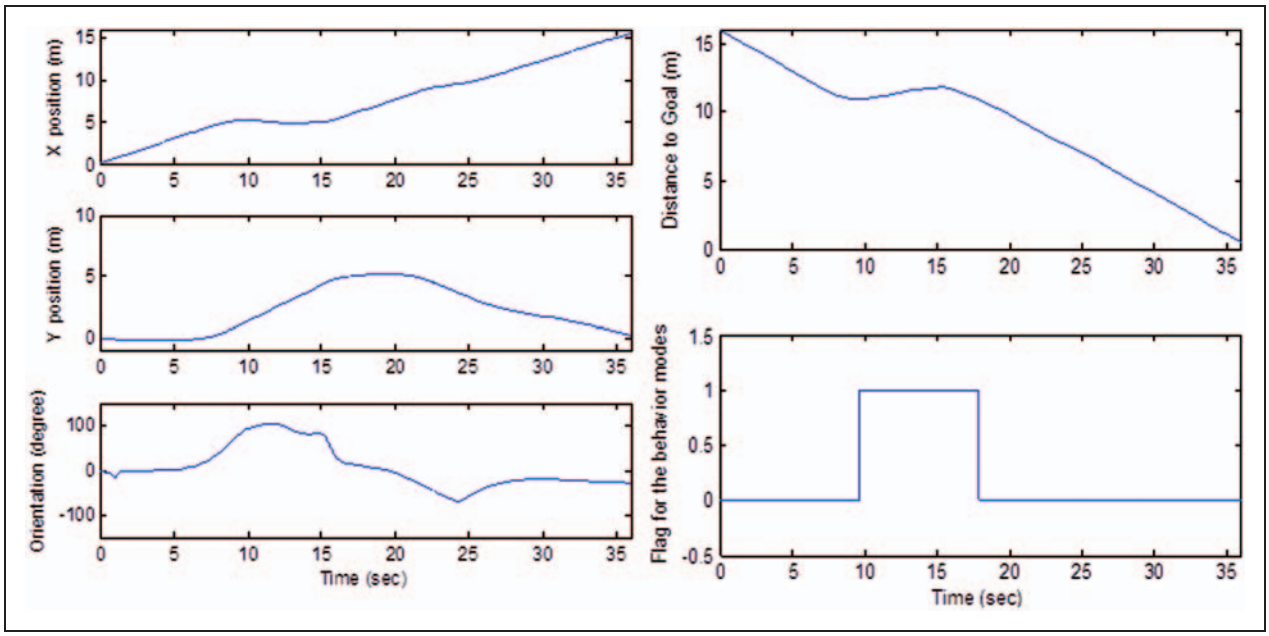

Figure 19. Robot configuration (left), the distance to goal and the flag of behavior modes (right) during the experiment of simple environment with $\theta(0) = 0$.

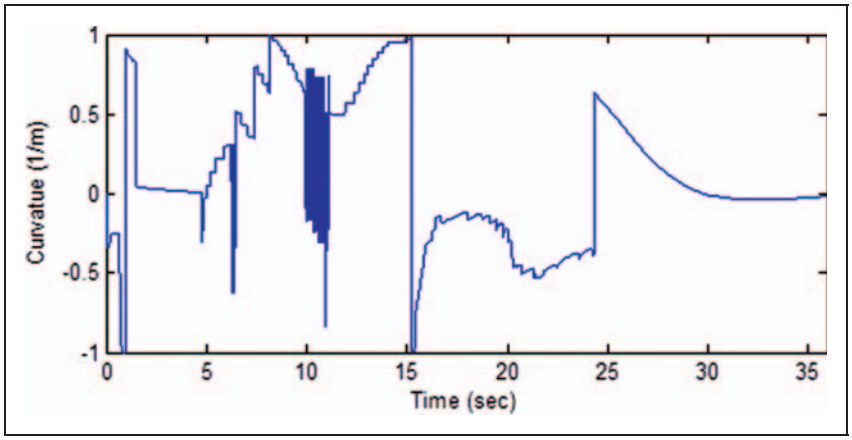

Figure 20. Curvature for the car-like robot during a simple environment with $\theta(0) = 0$.

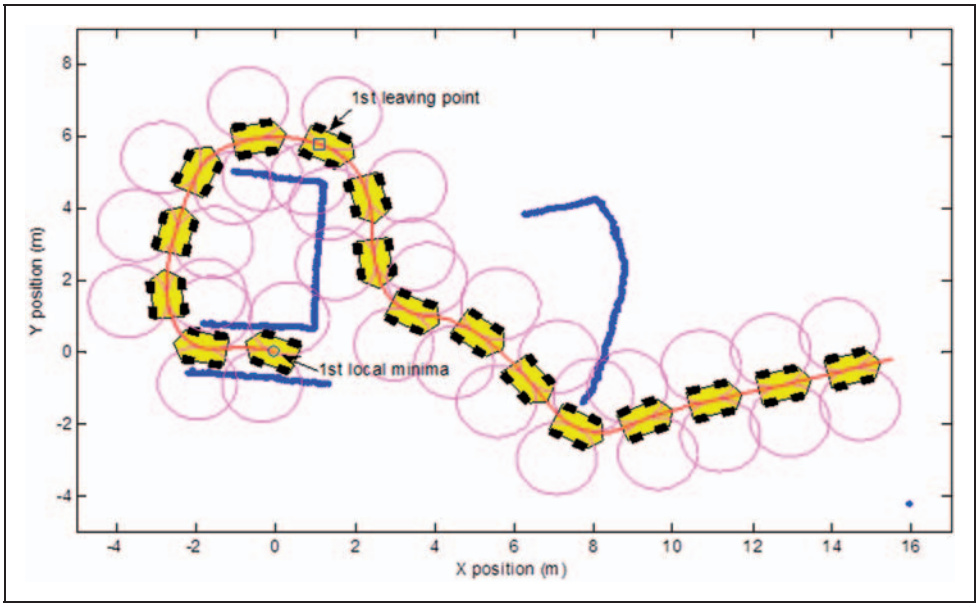

Figure 21. Experimental results in a simple environment when the initial orientation of the robot is 160°, $\theta(0) = 160°$.

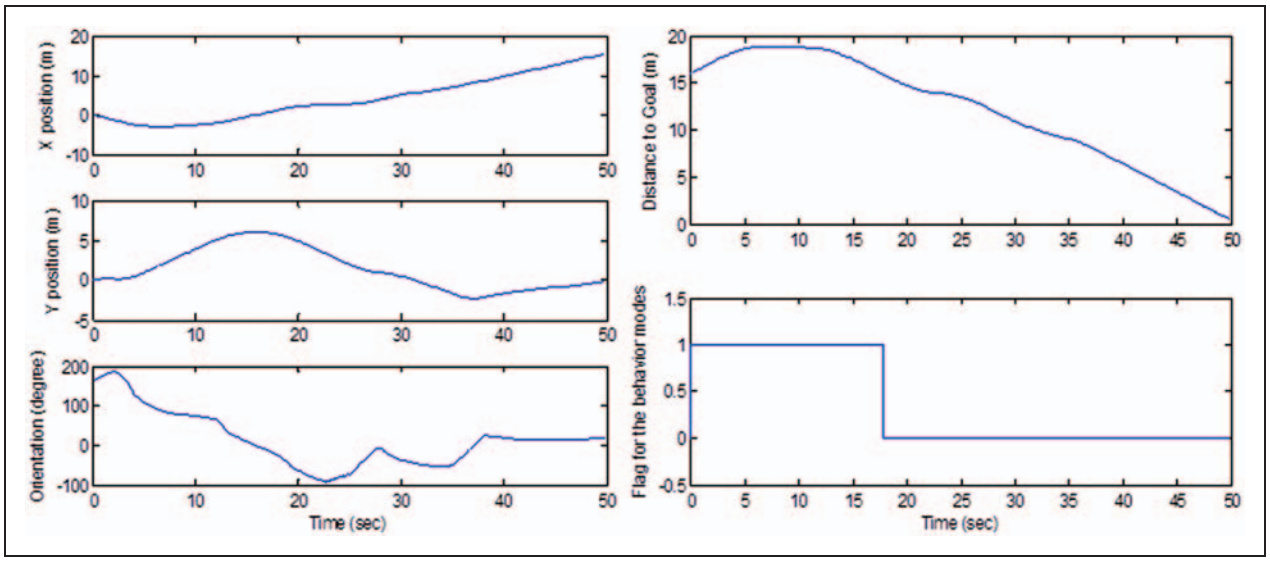

Figure 22. Robot configuration (left), the distance to goal and the flag of behavior modes (right) during the experiment in a simple environment with $\theta(0) = 160°$.

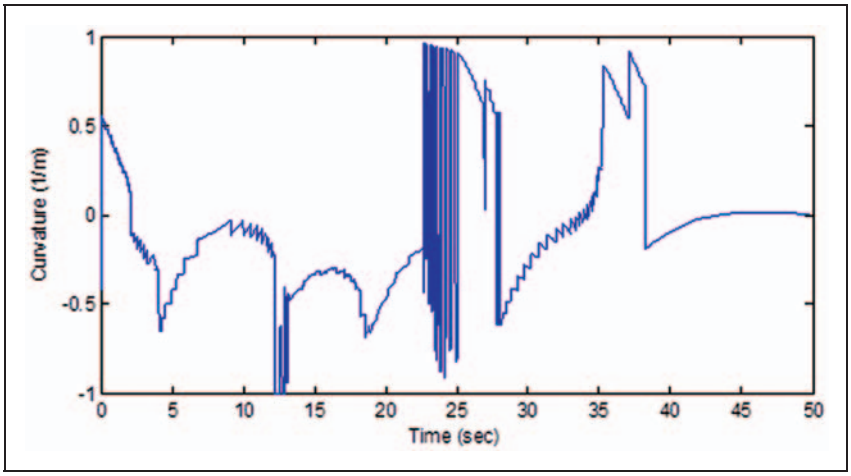

Figure 23. Curvature for the car-like robot during the experiment in a simple environment with $\theta(0) = 160°$.

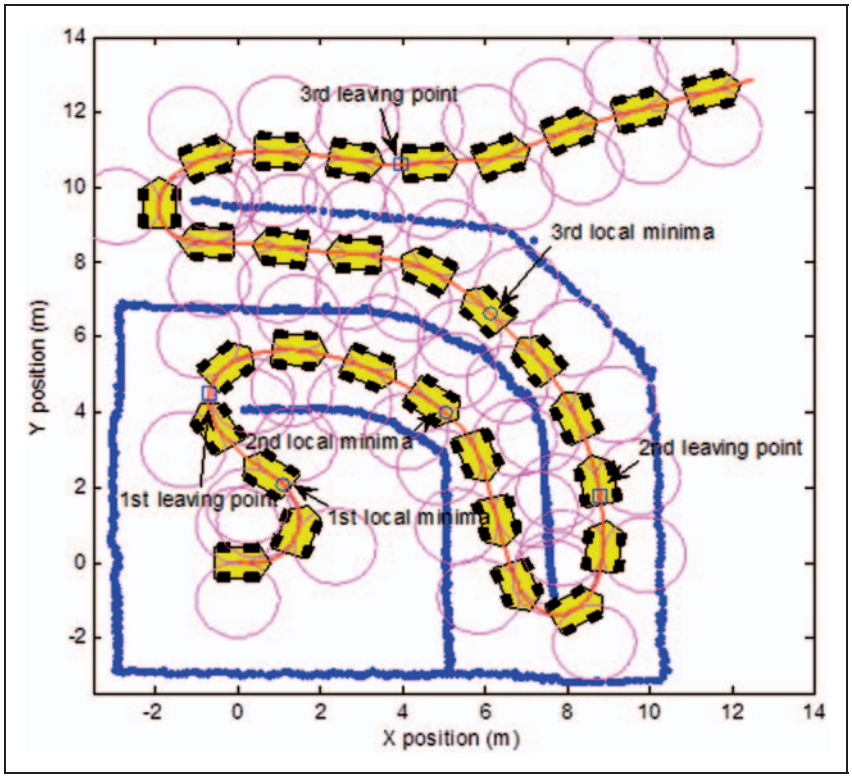

Figure 24. Experimental results in a maze-like environment.

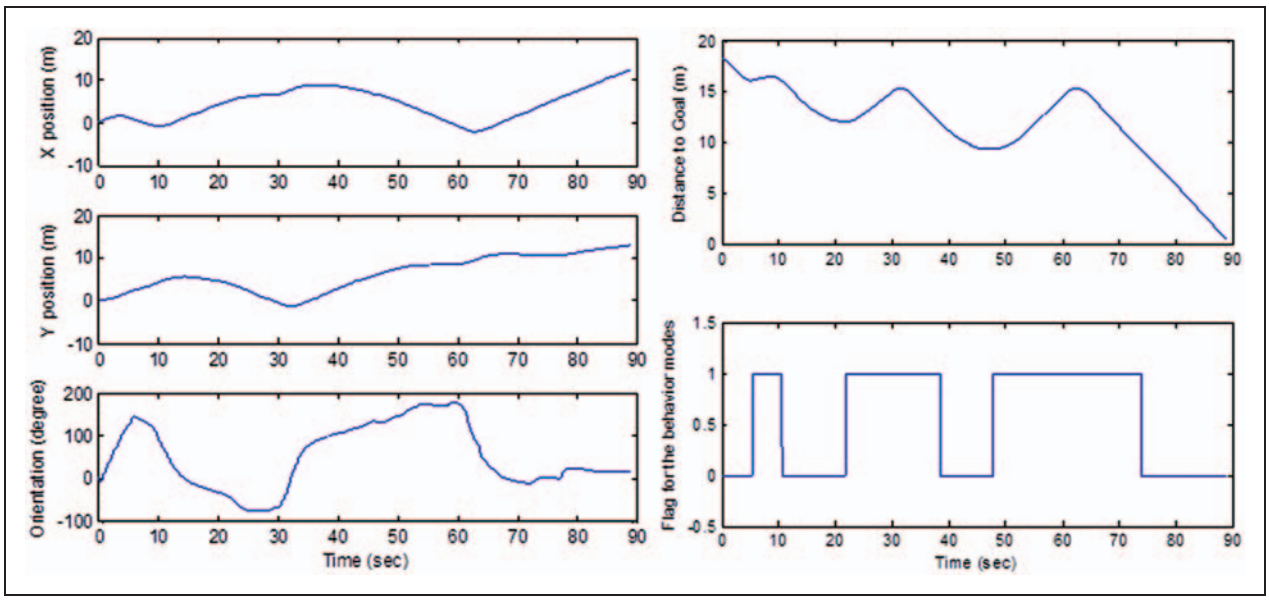

Figure 25. Robot configuration (left), the distance to goal and the flag for behavior modes (right) during the experiment in a maze-like environment.

obstacle point above 5 m because the robot need not react to the obstacle at a far distance. Thus, the maximum sensing range and angle for the range sensor are 5 m and 180°, respectively. If look-ahead distance for the robot is 2 m, then the minimum turning radius is determined as 1 m from equation (10). Although the PowerBot does not have a steering device, but a differential drive, we limit the velocity commands to the right and left wheels so that this robot has a minimum turning radius like a car-like robot. As the robot's system corrects the orientation using the gyro sensor and long-range navigation is not considered, we rely on the on-board odometry by ignoring the small amount of localization error.

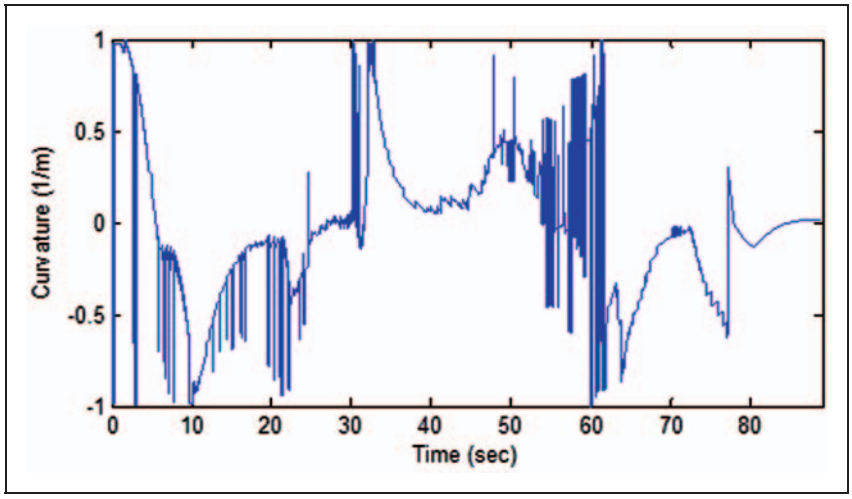

Figure 26. Curvature for the car-like robot during the experiment in a maze-like environment.

The first experiment includes two repetitions with a different initial orientation of the robot. Figure 18 shows the resultant trajectory from the initial configuration $\mathbf{q}(0) = [0\ 0\ 0]^T$ to $\mathbf{p}_G = [16\,\text{m}\ 0]^T$. The circles attached to the robot present the minimum size of turn. The robot's initial orientation is directed to the goal point, so the distance to the goal decreases at the beginning, as shown in Figure 19, where the flag = 0 is the motion-to-goal behavior and 1 the wall-following. Thus, we can see that only one local minima is determined. The curvature during the navigation is presented in Figure 20. Next, we repeat the same experiment except for the robot's initial orientation. The initial configuration of the robot is $\mathbf{q}(0) = [0\ 0\ 160^\circ]^T$. The shortest path to the goal at the start point is that the robot turns right with a minimum turning radius and goes straight; however, this nonholonomic path overlaps with the obstacles. Thus, the robot should start with wall-following because of the blocking obstacle at the start point. Figure 21 shows the experimental results for this case, and the graphs shown in Figures 22 and 23 are the robot configuration, the flag of the behavior mode, and the curvature, respectively.

We tested the navigation algorithm with a maze-like environment. For the second experiment, the initial configuration and the goal point is given by $\mathbf{q}(0) = [0\ 0\ 0]^T$ and $\mathbf{p}_G = [13\,\text{m}\ 13\,\text{m}]^T$. The number of local minima was three, as shown in Figure 24, and the corresponding results of robot configuration and the flag of the behavior mode is represented in Figure 25. The experiment takes about 90 s, and the curvature for the robot is calculated as shown in Figure 26.

Conclusions

This article introduces the navigation algorithm for a car-like robot. To be more practical, the car-like robot is equipped with a range sensor of a limited range and angle. The proposed navigation method is based on the classical Bug family algorithms which are the simplest path planners for robot navigation. Therefore, our study does not require a local path planner to obtain the nonholonomic path, thereby avoiding heavy computation requirements. Also, for nonholonomic mobile robot navigation, different from the previous navigation algorithms, the proposed algorithm provides instantaneous nonholonomic motion to the car-like robot in real time. The performance of the navigation algorithm is tested through several experiments. By testing the navigation algorithm with a simple environment and a complex maze-like environment, we have shown that the car-like robot can approach the goal point without violating the motion constraint.

Funding

This study was supported by the The Ministry of Knowledge Economy (MKE), Korea, under the 'Advanced Robot Manipulation Research Center' support program supervised by the NIPA (National IT Industry Promotion Agency), (grant no. NIPA-2012-H1502-12-1002) and also supported by the MKE, Korea, under the Industrial Foundation Technology Development Program supervised by the Korea Evaluation Institute of Industrial Technology (grant no. 10038660, Development of the control technology with sensor fusion based recognition for dual-arm work and the technology of manufacturing process with multi-robot cooperation).

References

1. Lumelsky VJ and Stepanov A. Path planning strategies for point automaton moving amidst unknown obstacles of arbitrary shape. *Algorithmica* 1987; 2: 403–430.
2. Kamon I, Rimon E and Rivlin E. Tangentbug: a range-sensor based navigation algorithm. *Int J Rob Res* 1998; 17: 934–953.

3. Laubach S and Burdick J. An autonomous sensor-based path planner for planetary microrovers. In: *Proceedings of IEEE ICRA*, Detroit, MI, 11–15 May 1999, pp.347–354.
4. Mastrogiovanni F, Sgorbissa A and Zaccaria R. Robust navigation in an unknown environment with minimal sensing and representation. *IEEE Trans Syst Man Cybern Part B: Cybern* 2009; 39(1): 212–219.
5. Magid E and Rivlin E. CAUTIOUSBUG: A competitive algorithm for sensory-based robot navigation. In: *Proceedings of IEEE/RSJ International Conference IROS*, Sendai, Japan, 28 September–2 October 2004, pp.2757–2762.
6. Gabriely Y and Rimon E. CBUG: a quadratically competitive mobile robot navigation algorithm. *IEEE Trans Rob* 2008; 24(6): 1451–1457.
7. Minguez J and Montano L. Extending collision avoidance methods to consider the vehicle shape, kinematics, and dynamics of a mobile robot. *IEEE Trans Rob* 2009; 25(2): 367–381.
8. Lamiraux F, Bonnafous D and Lefebvre O. Reactive path deformation for nonholonomic mobile robots. *IEEE Trans Rob* 2004; 20(6): 967–977.
9. Nagatani K, Iwai Y and Tanaka Y. Sensor-based navigation for car-like mobile robots based on a generalized Voronoi graph. *Adv Rob* 2003; 17(5): 385–401.
10. Lanzoni C and Sanchez A. Sensor-based motion planning for car-like mobile robots in unknown environments. In: *Proceedings of IEEE ICRA*, Taipei, Taiwan, 14–19 September 2003, pp.4258–4263.
11. Ollero A and Heredia G. Stability analysis of mobile robot path tracking. In: *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems*, Pittsburgh, PA, 5–9 August 1995, vol. 3, pp.461–466.
12. Hellström T and Ringdahl O. Follow the past – a path tracking algorithm for autonomous vehicles. *Int J Veh Auton Syst* 2006; 4: 216–224.